codefinity

Introduction to Data Engineering with Azure

Master the essentials of data engineering with Microsoft Azure in this comprehensive course. Starting with foundational concepts like cloud computing, resource management, and storage solutions, you'll progress to hands-on training in Azure Data Factory (ADF), mastering ETL/ELT workflows, and advanced data transformations. Through practical examples and real-world problem-solving, you'll gain the skills to design, implement, and optimize scalable data solutions using Azure.

©2025 Codefinity. All Terms Reserved.

Getting Started with Azure and Core Tools

©2025 Codefinity. All Terms Reserved.

codefinity

Getting Started

Welcome to the exciting journey of data engineering with Azure!

Before starting this course, I recommend you complete the following courses:

Prerequisites

- **Introduction to SQL**
- **Intermediate SQL**

Later in the course, we will be using statements covered in these courses, and if you're not familiar with these topics, it will be **difficult for you** to adapt and fully understand the material.

In this chapter, we'll lay the foundation by understanding the key concepts and answering some fundamental questions.

Let's now try to answer questions from the video using some real-life examples.

What is Data Engineering

Imagine you own a coffee shop chain, and you want to understand how to improve sales. Every day, your shops generate data: customer orders, sales trends, inventory levels, and even weather conditions that impact foot traffic. But this **data is scattered** — receipts at one store, spreadsheets from inventory systems, and temperature logs from sensors.

Data engineering is the **process of collecting, organizing, and preparing this raw data** so you can use it to answer questions like "Which coffee flavors are most popular in different locations?" or "How does rainy weather affect customer visits?".

©2025 Codefinity. All Terms Reserved.

A data engineer **designs systems to bring all this information together** and make it usable for decisions like these.

What is ETL/ELT?

Now, let's say you've decided to analyze the coffee sales data. The process of **ETL (Extract, Transform, Load)** is like running your coffee shop's nightly cleanup and prep:

- **Extract**: you collect the day's receipts, inventory logs, and weather reports from multiple locations;
- **Transform**: you clean up the receipts by removing duplicate entries, organize inventory logs into categories, and calculate averages for weather data. This step ensures the data is accurate and easy to analyze;
- **Load**: finally, you store the cleaned and organized data into a central system, like a database or a reporting dashboard, so you can use it to make informed decisions.

In **ELT (Extract, Load, Transform)**, you skip the middle step and load the raw data into a system like Azure first, then transform it there. This approach is better for large datasets because cloud tools can handle the heavy processing.

Why Use Azure?

Imagine your coffee shop chain **grows to 100 locations**. You're now dealing with massive amounts of data every day—orders, payments, inventory, and customer reviews. Storing and processing this data on local servers is not only expensive but also slow.

Azure solves this problem by offering **scalable, cloud-based tools** designed for businesses like yours.

With Azure, you **don't need to worry about running out of storage or processing power** as your chain expands. Plus, it's **cost-effective** since you only pay for what you use.

©2025 Codefinity. All Terms Reserved.

codefinity

📖 Understanding Azure Subscriptions

An **Azure subscription** is your entry ticket to the Microsoft Azure cloud platform. Think of it as a membership or account that gives you access to Azure's vast suite of services. Each subscription acts **as a container** for the resources you create, such as storage accounts, databases, or virtual networks. Within this container, you manage resources, track usage, and handle billing.

> **Definition**
>
> In Azure, a **resource** is any service or component that you can create and manage within your Azure subscription. Resources are the **building blocks of your cloud infrastructure**, and they are typically used to deliver specific functionality or services.

The Purpose of Azure Subscription

- **Access to services**: without a subscription, you can't use Azure services. It's like needing a library card to borrow books. Once you have a subscription, you can deploy and manage resources across Azure's global network;
- **Resource organization**: subscriptions help you group resources logically. For example, if you're running two projects (say, one for a coffee shop app and another for an inventory management system), you can separate their resources into different subscriptions for easier management;
- **Billing and budgeting**: every subscription has its own billing structure, so you can track costs for specific projects or teams. Azure provides tools to monitor usage and set spending limits within a subscription, ensuring you stay within budget.

©2025 Codefinity. All Terms Reserved.

codefinity

🕮 Organizing Resources with Resource Groups

Once you've set up your Azure subscription, the next step is to organize your resources. This is where **Azure Resource Groups** come in.

An **Azure Resource Group** is a collection of resources that share the same lifecycle. Resources in a resource group are deployed, updated, and deleted together. You can think of it like a project folder that contains all the necessary files for a specific task.

The Purpose of Azure Resource Groups

- **Simplified management**: by grouping related resources together, you can manage them as a unit. For example, if you want to delete all resources associated with a particular project, you can delete the entire resource group instead of deleting each individual resource;
- **Access control**: Azure allows you to apply **role-based access control (RBAC)** at the resource group level. This means you can assign specific permissions to users or groups to access or manage all resources within a group. For instance, you might allow a developer to manage resources in the "Development" resource group but restrict access to the "Production" group;
- **Resource Organization**: Resource groups allow you to logically organize resources according to the stage of your project, such as development, testing, and production environments.

©2025 Codefinity. All Terms Reserved.

codefinity

🕮 Using Azure Blob Storage for Data Storage

In this chapter, we will dive into **Azure Blob Storage**, one of the most powerful and commonly used services for storing data in the cloud.

Azure Blob Storage is a service provided by Microsoft Azure to store large amounts of unstructured data. The term "blob" stands for **Binary Large Object**, and it refers to the type of data stored in Azure. Blob Storage is ideal for storing data that doesn't have a predefined model or structure, such as: text files, audio or video files, images etc.

Azure Data Lake Storage Gen2 is an enhanced version of Azure Blob Storage that provides **hierarchical namespace support**, which allows you to organize data more effectively and makes it easier to manage large datasets.

Key Features of Azure Data Lake Storage Gen 2 (ADLS Gen2)

- **Scalability**: ADLS Gen2 is built to handle massive amounts of data. Whether you're working with gigabytes or petabytes, you can store and manage it all within the same account;
- **Hierarchical namespace**: ADLS Gen2 supports a hierarchical namespace that allows you to organize data in a directory structure similar to a file system. This makes it easier to manage large sets of files, apply access control policies, and optimize the performance of data operations;
- **Cost efficiency**: Azure offers different tiers (Hot, Cool, and Archive) that allow you to store your data in the most cost-effective manner depending on how frequently you access it.

©2025 Codefinity. All Terms Reserved.

codefinity

🕮 Setting Up Azure SQL Database

Azure SQL Database is a Platform-as-a-Service (PaaS) offering that provides fully managed SQL databases in the Azure ecosystem. It eliminates the need to manage hardware, perform manual updates, or handle complex backups.

> **Definition**
>
> **Platform as a Service (PaaS)** is a cloud computing model that provides a managed platform for building, deploying, and running applications without the need to manage underlying infrastructure. It offers **pre-configured resources** like operating systems, runtime environments, development tools, and middleware, allowing developers to focus on coding and application logic rather than managing servers or storage.

You will also need to use another important resource for your data engineering tasks - SQL Server.

In Azure, an **SQL Server** is a logical container for managing your Azure SQL Databases. It doesn't represent a physical server but instead acts as a **management layer to handle connections, configurations, and security for databases within it**.

Think of it as a gatekeeper: you create a server in Azure, and all your Azure SQL Databases under that server share common settings, like administrative credentials, firewalls, and connection strings.

Why Use Azure SQL Database in Data Engineering?

In data engineering, **structured data** plays a critical role, particularly in transactional systems, reporting, and analytics pipelines. Azure SQL Database is perfect for:

©2025 Codefinity. All Terms Reserved.

- Storing **transactional data** from applications;
- **Supporting ETL/ELT workflows** with tools like Azure Data Factory;
- Enabling **data transformation and reporting** with SQL queries and views.

For example, if you're tracking customer purchases, Azure SQL Database can store customer profiles, transaction details, and inventory records in relational tables.

©2025 Codefinity. All Terms Reserved.

codefinity

📖 Managing Credentials with Azure Key Vault

In data engineering, **managing credentials securely** is critical to ensure that sensitive information such as API keys, database connection strings, and passwords are not exposed. Azure Key Vault is a robust solution to help you achieve this.

Azure Key Vault is a cloud service designed to store and manage sensitive information securely. It acts as a centralized repository for secrets and keys.

> **Definition**
>
> **Secrets** are secure strings like API keys or database credentials.
>
> **Keys** are cryptographic keys used for data encryption and signing.

Key Vault ensures that sensitive information is protected through encryption and access controls, eliminating the risks of hardcoding credentials in applications.

Why Use Azure Key Vault?

- **Enhanced security**: secrets are encrypted with strong algorithms and stored securely;
- **Centralized management**: manage all your sensitive information in one place;
- **Access control**: only authorized applications and users can access the stored secrets.

For example, instead of embedding a database password in your code, you store it in Azure Key Vault. Your application retrieves it at runtime securely.

©2025 Codefinity. All Terms Reserved.

codefinity

🕮 Introduction to Azure Data Factory

Modern data engineering often involves moving and transforming vast amounts of data from various sources to destinations, ensuring it's ready for analytics or business applications. This is where **Azure Data Factory (ADF)** comes in - a powerful, cloud-based data integration service designed to simplify and automate these tasks.

Azure Data Factory (ADF) is a Platform as a Service (PaaS) offering from Microsoft that enables you to create, manage, and monitor ETL/ELT data pipelines. These pipelines orchestrate workflows to: - **Extract data** from diverse sources like on-premises databases, APIs, and cloud storage; - **Transform the data** by cleansing, aggregating, or enriching it; - **Load it into destinations** such as Azure SQL Database, Data Lakes, or Power BI.

Key Components of Azure Data Factory

- **Pipelines**: the core unit in ADF where you define the sequence of tasks (activities) to process your data;
- **Datasets**: represent your data structure, such as a file in Blob Storage or a table in a database;
- **Linked Services**: define the connection details to your data sources and destinations (e.g., credentials, endpoints).

Why Use Azure Data Factory?

©2025 Codefinity. All Terms Reserved.

- **Seamless data movement**: connects to over 90 data sources, both cloud-based and on-premises;
- **No-Code/Low-Code development**: build pipelines visually using a drag-and-drop interface or define workflows in code;
- **Scalability**: automatically scales to handle large datasets and high-frequency data transfers;
- **Cost-Effective**: pay-as-you-go pricing with no upfront costs.

©2025 Codefinity. All Terms Reserved.

codefinity

Foundations of Azure Data Factory

©2025 Codefinity. All Terms Reserved.

codefinity

📖 Linked Services

In Azure Data Factory, one of the first steps in building a pipeline is to **establish connections** to your data sources and destinations. These connections are defined using Linked Services, which act as the bridge between ADF and the data you need to work with.

> **Definition**
>
> A **Linked Service** in ADF is similar to a connection string in traditional applications. It **defines the connection details** for a specific data store or compute service, such as credentials, endpoints, and configuration settings.

For example, if your pipeline needs to move data from Azure Blob Storage to Azure SQL Database, you'll create three Linked Services: one for Azure Blob Storage, one for Azure SQL Database and one for Key Vault to store credentials.

Why Are Linked Services Important?

- **Centralized Connection Management**: instead of specifying connection details repeatedly, you define them once and reuse them across multiple datasets and activities;
- **Secure access**: credentials and sensitive information are securely stored and can be encrypted or integrated with Azure Key Vault for enhanced security;
- **Flexibility**: ADF supports over 90 different Linked Services, enabling seamless integration with diverse systems, including cloud-based, on-premises, and SaaS platforms.

©2025 Codefinity. All Terms Reserved.

codefinity

Datasets

When working with data in Azure Data Factory, datasets play a crucial role. They define the data structures or schema used in pipelines, essentially serving as a **pointer to the data you want to move or transform**. Datasets work in tandem with Linked Services to help ADF access, process, and manage data efficiently.

> **Definition**
>
> Formally, a **dataset** in ADF represents a specific dataset or table-like structure within a data source or destination. For instance, a dataset **could be a folder or file in Azure Blob Storage**, or it could also **represent a table** in Azure SQL Database.

While Linked Services define where the data resides, datasets specify what part of the data to work with.

Covid Dataset

Table Creation Script

Why Are Datasets Important?

- **Data interaction**: datasets provide the schema and parameters that help ADF understand and interact with your data;
- **Reusability**: you can define datasets once and reuse them across multiple activities in pipelines;
- **Flexibility**: datasets allow dynamic configurations using parameters, enabling pipelines to handle variable data inputs or outputs.

©2025 Codefinity. All Terms Reserved.

codefinity

Note

In this chapter, we created datasets that represent our data in Azure Storage and Azure Database without any parameterizations.

©2025 Codefinity. All Terms Reserved.

codefinity

What is ADF Pipeline?

In Azure Data Factory, **pipelines** are the backbone of data integration workflows. They act as **containers for activities**, defining the sequence of tasks needed to move, transform, and process data. Pipelines streamline the process of working with diverse data sources, enabling seamless integration and automation of tasks. Think of it as a **recipe for data movement and transformation**, where each activity represents a step in the process.

> **Definition**
>
> An **ADF Pipeline** is a logical container for grouping together activities that perform tasks such as data movement, transformation, and control flow. It defines the sequence of operations to extract, transform, and load (ETL) data from source to destination.

In the next chapters, we will consider the most important pipeline activities and now let's start with the most commonly used - **Copy Activity**.

What is Copy Activity?

The **Copy Activity** is like a delivery truck for data. It extracts data from a source (e.g., Azure Blob Storage, Amazon S3, or an on-premises database) and loads it into a destination (e.g., Azure SQL Database or Data Lake). It handles the heavy lifting of data transfer while ensuring accuracy and efficiency.

> **Definition**
>
> A **Copy Activity** in Azure Data Factory is used to copy data from a source to a destination within a pipeline. It handles the extraction, transfer, and loading of data between various data stores.

©2025 Codefinity. All Terms Reserved.

codefinity

How to Create a Copy Activity

1. Create a new pipeline in the Author section of Azure Data Factory Studio;
2. Drag the **Copy Data Activity** from the Activities pane onto the pipeline canvas;
3. Configure the **source** by selecting an existing Linked Service and Dataset for the data you want to move. Optionally, apply filters or specify queries to extract specific data using **Query** option;
4. Configure the **destination (Sink)** by choosing a Linked Service and Dataset where the data will be stored;
5. Align source and destination fields in the Mapping tab if necessary;
6. Validate the pipeline by hitting **Debug** button.

©2025 Codefinity. All Terms Reserved.

codefinity

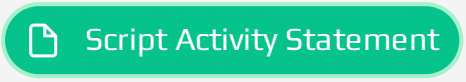
Script Activity

The **Script Activity** in Azure Data Factory is a powerful tool that allows you to execute SQL scripts as part of your data pipelines. This feature is particularly useful for tasks such as managing database schemas, performing updates, or running any custom SQL commands needed to prepare or manage your data infrastructure.

Definition

A **Script Activity** in Azure Data Factory enables you to **connect to a database and execute pre-defined SQL statements** directly from a pipeline.

Unlike the Copy Activity, which is focused on moving data, the Script Activity is designed to **modify, update, or interact with the database itself**. It is commonly used in scenarios such as adding or removing columns, creating or dropping tables, or applying business rules through stored procedures.



How to Use Script Activity

1. Start by dragging the **Script Activity** from the Activities pane onto the pipeline canvas in Azure Data Factory Studio;
2. Link the Script Activity to your database by selecting a pre-configured **Linked Service**;
3. Open the **Script** tab and write your SQL commands, such as creating, modifying, or dropping database objects;
4. Validate the pipeline.

©2025 Codefinity. All Terms Reserved.

codefinity

Lookup Activity

The **Lookup Activity** in Azure Data Factory is a versatile tool that retrieves data from a specified source, such as a database, blob storage, or a data lake. It allows you to **fetch single or multiple rows of data** to use dynamically in your pipelines. This makes it ideal for use cases like conditional processing, dynamic configuration, or filtering data based on specific patterns.

Definition

A **Lookup Activity** in Azure Data Factory retrieves **data from a specified source** and allows its output to be used dynamically in subsequent pipeline activities.

It connects to a data source and executes a query to fetch data. The output can be used as **input for other activities in the pipeline**, enabling dynamic and flexible workflows. For example, you can retrieve a list of regions from a database and use them to trigger region-specific tasks.

Lookup Activity Statement

How to Use Lookup Activity

©2025 Codefinity. All Terms Reserved.

1. **Drag** the Lookup Activity onto your pipeline canvas in Azure Data Factory Studio;
2. **Link** the activity to a Linked Service that points to your data source;
3. In the Source Dataset field, **select or create a dataset** that represents your data source;
4. Configure the **Query field** by writing a SQL query (for databases) or specifying a file path/pattern (for files);
5. Decide whether to retrieve a single row or multiple rows by adjusting the **First Row Only** option in the activity settings;
6. Use the output dynamically in subsequent activities by referencing it with expressions like `@activity('LookupActivityName').output.value`.

When you configure the Lookup Activity in Azure Data Factory with the **"Use First Row Only"** option, it ensures that only the first row of the query result is retrieved. This setup is particularly useful when you need a single data point or record to drive subsequent activities in your pipeline.

Once the Lookup Activity is configured, you can reference its first-row output in subsequent activities using the following expression: `@activity('LookupActivityName').output.firstRow`.

©2025 Codefinity. All Terms Reserved.

codefinity

Dataset Parameters

Parameters in Azure Data Factory are **dynamic elements that make pipelines, datasets, and linked services flexible and reusable**. By leveraging parameters, you can create solutions that adapt to different scenarios with minimal configuration changes. In this chapter, we'll focus on dataset parameters, particularly using them to **define the table name dynamically** for a SQL dataset.

Definition

Dataset Parameters enable Azure Data Factory datasets to adapt dynamically at runtime by using parameters, allowing tasks such as defining table names or file paths to be flexible and reusable.

For example, in a SQL dataset, **instead of hardcoding the table name**, you can use a parameter to set the table name at runtime. This approach is especially useful when working with multiple tables or environments.

We created a dataset with a **table name parameter** to enable dynamic table selection during data operations. By defining a parameter for the table name in a SQL dataset, you can reuse the same dataset for multiple tables, making your workflows more efficient and adaptable.

This setup lays the foundation for the **ForEach Activity**, where we will dynamically iterate over multiple table names and perform operations on them. This approach is essential for scalable and flexible data integration pipelines.

©2025 Codefinity. All Terms Reserved.

codefinity

Note

You can also create parameters for pipelines and some other activities, but they are outside the scope of this course. Feel free to experiment and try them on your own.

©2025 Codefinity. All Terms Reserved.

codefinity

ForEach Activity

The **ForEach activity** in Azure Data Factory is a powerful tool that enables you to perform **iterative operations**, such as processing multiple files or iterating through a list of items.

Definition

ForEach Activity in Azure Data Factory acts as a loop, iterating over a collection of items, such as file names or table names, and performing specified actions like transforming or copying data for each item.

The **Lookup activity is often used in combination with the ForEach activity** to retrieve a list of items, such as file names or table names, for iteration. By passing the Lookup output to the ForEach activity, you can dynamically process each item in the collection, enabling scalable and flexible workflows in Azure Data Factory.

ForEach Activity Statement

How to Use ForEach Activity

©2025 Codefinity. All Terms Reserved.

1. Start by using a **Lookup activity** to retrieve data;
2. Drag the **ForEach Activity** from the Activities pane onto the pipeline canvas;
3. In the **Settings** tab of the ForEach Activity, set the **Items** field to the output of the Lookup activity, such as: `@activity('LookupActivityName').output.value`;
4. Inside the ForEach Activity, add a **Copy Data Activity** or **any other activity** to process each item in the list dynamically;
5. In the **Source** tab of the Copy Data Activity, configure the input dataset for the data you want to process and use dynamic content to reference the current iteration. In our case we use the following query: `SELECT * FROM CovidData WHERE "HHSRegion" = '@{item().HHSRegion}'`;
6. In the **Sink** tab of the Copy Data Activity, configure the destination table name dynamically using the expression: `TableName_@{item().ColumnName}`;
7. Validate and test the pipeline to ensure that data is correctly processed for each item in the dataset.

@item() expression

In Azure Data Factory (ADF), the `@item()` expression is used inside the ForEach activity to **reference the current item** in the loop as it iterates through the collection of items. The `@item()` expression allows you to dynamically access data or values from the current iteration and use them in subsequent activities within the ForEach loop.

Its attributes refer to the **properties of the item being iterated over**. For example, if the loop is iterating through a list of objects, `@item()` can access specific attributes like `@item().Region` or `@item().FileName`, allowing dynamic referencing of data for each iteration.

©2025 Codefinity. All Terms Reserved.

codefinity

Note

In Azure Data Factory, you can use the `replace()` function to manipulate strings by replacing specific characters or substrings. For example, to replace all spaces (' ') with underscores ('_'), you would use the following expression:

```
replace('your string here', ' ', '_')
```

This function helps in cleaning and standardizing data, particularly when working with file names or table names that may contain spaces, making them easier to process programmatically.

©2025 Codefinity. All Terms Reserved.

codefinity

📖 Execute Pipeline Activity

Now we will learn about the **Execute Pipeline Activity** in Azure Data Factory. This activity is essential when you need to **chain pipelines together and execute them in a sequence**, ensuring a smooth workflow from one process to another.

> **Definition**
>
> **Execute Pipeline Activity** in Azure Data Factory allows one pipeline to invoke and execute another pipeline, enabling modular, reusable, and sequential workflows.

This activity allows you to **call and run another pipeline within a pipeline**. It helps you break down complex workflows into smaller, modular components, which can then be executed in a sequence or based on certain conditions. This is particularly useful when different stages of data processing or transformation need to be carried out independently but in a specific order.

Why Do We Need the Execute Pipeline Activity?

In our case, the Execute Pipeline Activity was used to connect two pipelines: one for loading data into the database and another for creating separate tables for each region. This activity is useful for several reasons, helping to improve the structure and efficiency of data workflows in Azure Data Factory.

Modularity and Reusability

The Execute Pipeline Activity enables the creation of modular, reusable pipelines. Instead of duplicating logic in multiple pipelines, you can design a single pipeline for specific tasks (e.g., data ingestion, transformation) and reference it across different pipelines whenever needed.

©2025 Codefinity. All Terms Reserved.

Chaining Pipelines Together

Data engineering workflows often require a series of operations to be executed in sequence. The Execute Pipeline Activity allows you to chain dependent pipelines, ensuring they run one after the other. For example, one pipeline may handle data extraction, and once it completes, another pipeline can trigger transformation and then load the data to its destination.

Simplifying Complex Workflows

Complex workflows are easier to manage when broken into smaller, independent pipelines. The Execute Pipeline Activity facilitates the orchestration of these smaller pipelines, making the entire workflow easier to maintain, debug, and optimize.

©2025 Codefinity. All Terms Reserved.

codefinity

Data Flows and Transformations in ADF

©2025 Codefinity. All Terms Reserved.

codefinity

Understanding Data Flows in Azure Data Factory

Definition

Data Flows in Azure Data Factory are visual, code-free tools that enable complex data transformations within ETL or ELT workflows directly in ADF.

For instance, imagine a scenario where you need to clean, enrich, and aggregate sales data from multiple regions. Instead of writing extensive SQL or Python scripts, you can use a Data Flow to visually map these transformations and execute them seamlessly within ADF.

Key Components of Data Flows

- **Source Transformation**: defines where the data originates, such as Blob Storage or a SQL Database;
- **Transformations**: include tools like filtering, joining, aggregating, or deriving new columns to manipulate the data;
- **Sink Transformation**: specifies the destination for the processed data, such as another SQL Database, a data lake, or a file storage.

We will start our work with creating simple dataflow with source and sink transformations.

Dataflow Source Statement

How to Set Up a Source Transformation

©2025 Codefinity. All Terms Reserved.

1. Add a **new Data Flow** in the Author section of Azure Data Factory Studio;
2. Drag a **Source Transformation** from the toolbox onto the Data Flow canvas;
3. In the **Source Transformation settings**, select a **Linked Service**, such as Azure SQL Database or Azure Blob Storage, to connect to your data source;
4. Choose an **existing Dataset** or create a **new Dataset** that represents the data to be ingested;
5. Configure **file format options** if connecting to Blob Storage, or provide a **SQL query** to filter or structure the incoming data for databases;
6. **Validate the configuration** and **preview the data** to ensure the source is correctly set up.

Sink Transformation for Processed Data

After defining transformations, use a Sink Transformation to specify where the transformed data will be stored. For example, you might save aggregated data back to the SQL database or export it as a CSV file to Blob Storage.

> **Note**
>
> Remember, every dataflow **must include at least one** source transformation and one sink transformation.

©2025 Codefinity. All Terms Reserved.

codefinity

🕮 Derived Column Transformation

Derived columns allow you to enhance your data **by creating new fields or modifying existing ones** based on business logic. This functionality is vital for enriching your dataset to better serve downstream processes such as analytics, reporting, or machine learning.

> **Definition**
>
> A **Derived Column** is a new data field generated within a data flow by applying transformations or expressions to existing data.

For instance, you might create a `FullName` field by concatenating first and last names or derive a `DiscountedPrice` based on an existing `Price` field and a discount rate.

How to Create Derived Columns in ADF

1. Add a new Data Flow in the Author section of Azure Data Factory Studio;
2. Drag a **Derived Column Transformation** from the toolbox onto the Data Flow canvas and connect it to your data source;
3. In the **Derived Column Transformation** settings, define the column name for the new or modified field, such as `DeathRate`;
4. Write an expression using ADF's expression language. For example, to calculate a death rate: `iif(TotalDeaths > 0, (Covid19Deaths * 100.0) / TotalDeaths, 0.0)`;
5. Validate the configuration by previewing the data to ensure the derived column is calculated correctly;
6. Connect the transformed data to a **Sink Transformation** to send it to a destination, such as a SQL table or Blob Storage, for further processing.

©2025 Codefinity. All Terms Reserved.

You can learn more about ADF Expression Language using Official ADF Documentation.

©2025 Codefinity. All Terms Reserved.

codefinity

Combining Data with Joins

Data often resides in multiple tables or sources, making it essential to **merge information** to derive meaningful insights. In this chapter, you'll learn how to use joins in Azure Data Factory (ADF) Data Flows to merge datasets efficiently

Definition

Joins in ADF allow you to **combine data from two or more sources** based on a shared key or condition. This is particularly useful for **merging relational tables** or integrating data from different origins into a cohesive dataset.

ADF supports several types of joins:

- **Inner Join**: combines rows where keys match in both datasets;
- **Left Outer Join**: includes all rows from the left dataset and matching rows from the right;
- **Right Outer Join**: includes all rows from the right dataset and matching rows from the left;
- **Full Outer Join**: includes all rows from both datasets, with nulls for unmatched rows;
- **Cross Join**: produces a Cartesian product of both datasets.

Population Data Script

How to Combine Data with Joins in ADF

©2025 Codefinity. All Terms Reserved.

1. Create **new** Data Flow or use **existing** one;
2. Drag two **Source Transformations** onto the Data Flow canvas and connect them to the respective SQL tables;
3. Drag a **Join Transformation** from the toolbox onto the canvas and connect the two sources to it;
4. In the Join Transformation settings, select the **Join Type** and set the **Join Condition**;
5. Add a **Derived Column Transformation** or **any other transformation** after the join to get some insights;
6. Add a **Sink Transformation** to store the output;
7. Validate the Data Flow configuration to ensure everything is correct.

©2025 Codefinity. All Terms Reserved.

codefinity

Using Conditional Split and Filtering in ADF

This chapter explores how to use **Conditional Split** and **Filter Transformations** in Azure Data Factory (ADF) Data Flows to organize and refine data for downstream processes. You'll learn how to divide data based on conditions and filter out unwanted records efficiently.

> **Definition**
>
> A **Conditional Split** is a transformation that divides data into multiple streams based on specified conditions. It allows routing of records to different outputs for separate processing.

In Azure Data Factory, a **Conditional Split** transformation can be used to route records based on conditions, such as splitting data into "High" and "Low" categories based on a sales amount. For example, if the sales amount is greater than 1000, the record is sent to the "High Sales" output, otherwise, it's sent to the "Low Sales" output for further processing.

> **Definition**
>
> A **Filter** is a transformation that removes records according to specified conditions to clean the dataset before loading it into a database.

For example, if you want to filter out records with null or invalid email addresses, you can apply a filter that removes records where the email is either null or does not match a valid email format.

How to Use Conditional Split and Filter Transformations in ADF

©2025 Codefinity. All Terms Reserved.

1. **Create a new Data Flow** or use an **existing** one in the Author section of Azure Data Factory Studio;
2. **Drag a Source Transformation** onto the Data Flow canvas and configure it to ingest data, such as from SQL tables or Blob Storage;
3. **Add a Conditional Split Transformation** from the toolbox and connect it to your data source;
4. In the Conditional Split settings, **define conditions** to split the data into multiple streams. In our case, we used the following:
5. `LowRisk`: DeathRate < 5;
6. `HighRisk`: DeathRate > 10;
7. `MediumRisk`: DeathRate >= 5 and <= 10;
8. **Add a Filter Transformation** and connect it to the data stream that you want to filter;
9. In the Filter settings, define the **filter condition** to keep only necessary records. In our case we remained only records where the `WeekEndingDate` is after/ before `'2021-09-01'`;
10. **Connect each output stream** to separate **Sink Transformations** to store the split data in different destinations (e.g., one for low risk, one for high risk, and one for medium risk);
11. **Validate the Data Flow** configuration to ensure everything works correctly.

©2025 Codefinity. All Terms Reserved.

codefinity

🕮 Grouping, Select, and Sort Transformations in ADF

Now we will explore three essential transformations in Azure Data Factory: **Grouping, Select, and Sort**. These transformations are critical for organizing and structuring your data in the data flow. We will discuss how to use each transformation to manipulate data effectively.

> **Definition**
>
> The **Grouping transformation** in Azure Data Factory aggregates data by one or more grouping columns, enabling the application of aggregate functions such as sum, average, or count on grouped datasets.

For example, you can group sales data by the `"Region"` column and calculate the sum of `"SalesAmount"` to determine the total sales for each region. Similarly, grouping by `"ProductCategory"` and applying a count function can provide the number of products sold in each category.

> **Definition**
>
> The **Select transformation** in Azure Data Factory allows you to shape and restructure data by selecting, renaming, or dropping columns, functioning similarly to a projection operation in SQL.

For example, you can use the Select transformation to rename the column `"Cust_ID"` to `"CustomerID"` for better clarity. Additionally, you can drop unnecessary columns, such as `"TempData,"` to streamline the dataset for further analysis.

©2025 Codefinity. All Terms Reserved.

codefinity

Definition

The **Sort transformation** arranges data in ascending or descending order based on one or more columns, making it particularly useful for ordering data before further processing or for downstream systems that require sorted input.

For example, the Sort transformation can be used to arrange sales data in descending order based on the `"TotalSales"` column, ensuring the highest sales appear first. Alternatively, you could sort employee records by `"HireDate"` in ascending order to view the most recently hired employees.

How to Use Grouping, Select, and Sort Transformations in ADF

1. Create **new** Data Flow or use **existing** one;
2. Drag a **Grouping Transformation** onto the Data Flow canvas;
3. In the Grouping settings, choose the column(s) to group by (e.g., `Region`);
4. Define aggregation logic for other columns (e.g., sum, average, max);
5. Drag a **Select Transformation** onto the canvas;
6. In the Select settings, choose columns to keep, rename or reorder them as needed;
7. Drag a **Sort Transformation** onto the canvas;
8. In the Sort settings, define the column(s) to sort by and choose the sort order (ascending or descending);
9. Add a **Sink Transformation** to store the output in a destination like SQL or Blob Storage;
10. Validate the Data Flow configuration to ensure everything is correct.

©2025 Codefinity. All Terms Reserved.

codefinity

Note

You can adjust the order of the Group, Select, and Sort transformations within the Data Flow to better suit your business logic.

©2025 Codefinity. All Terms Reserved.

codefinity

Practical Problem Solving with ADF

©2025 Codefinity. All Terms Reserved.

Challenge: Loading Data to Several Tables

In this task, we will work with credit card data, with the main goal of loading this data into Azure while meeting all necessary requirements.

Imagine you are working for a bank that handles a significant volume of credit card data. Your team has been tasked with organizing this data into a format that can be easily analyzed based on card types. You've been provided with a dataset containing information about various cards, including both debit and credit cards. Your job is to **load this dataset into an Azure SQL database** and ensure that the card information is stored in separate tables for each card type: one table for credit cards, another for debit cards, and so on.

The main dataset looks as follows:

> **Note**
>
> All tables below are scrollable. Use the scroll bar to view all columns in each table.

©2025 Codefinity. All Terms Reserved.

codefinity

id	client_id	card_brand	card_type	card_number	e
4524	825	Visa	Debit	4344676511950444	1
2731	825	Visa	Debit	4956965974959986	1
3701	825	Visa	Debit	4582313478255491	0
42	825	Visa	Credit	4879494103069057	0
4659	825	Mastercard	Debit (Prepaid)	5722874738736011	0
4537	1746	Visa	Credit	4404898874682993	0
1278	1746	Visa	Debit	4001482973848631	0
3687	1746	Mastercard	Debit	5627220683410948	0
3465	1746	Mastercard	Debit (Prepaid)	5711382187309326	1
3754	1746	Mastercard	Debit (Prepaid)	5766121508358701	0
5144	1718	Mastercard	Debit	5495199163052054	0
2029	1718	Mastercard	Debit	5804499644308599	0

©2025 Codefinity. All Terms Reserved.

2379	1718	Mastercard	Debit	5766352389579834	0

Cards Dataset

This task involves:

- Loading data from CSV files to the cloud;
- Separating card data into distinct tables based on the card type;
- Ensuring the data is properly formatted for future analysis.

The resulting tables will look as follows.

Credit Cards Table

id	client_id	card_brand	card_type	card_number	e
42	825	Visa	Credit	4879494103069057	08
4537	1746	Visa	Credit	4404898874682993	09

Debit Cards Table

©2025 Codefinity. All Terms Reserved.

id	client_id	card_brand	card_type	card_number	e
4524	825	Visa	Debit	4344676511950444	1
2731	825	Visa	Debit	4956965974959986	1
3701	825	Visa	Debit	4582313478255491	0
1278	1746	Visa	Debit	4001482973848631	0
3687	1746	Mastercard	Debit	5627220683410948	0
5144	1718	Mastercard	Debit	5495199163052054	0
2379	1718	Mastercard	Debit	5766352389579834	0
2029	1718	Mastercard	Debit	5804499644308599	0

Debit Cards (Prepaid) Table

©2025 Codefinity. All Terms Reserved.

id	client_id	card_brand	card_type	card_number	exp
4659	825	Mastercard	Debit (Prepaid)	5722874738736011	03/
3465	1746	Mastercard	Debit (Prepaid)	5711382187309326	11/2
3754	1746	Mastercard	Debit (Prepaid)	5766121508358701	02/

Please note that the dataset may contain more than three card types, so you need to create separate tables for each of them!

Hint

To solve this task, you can use the materials from the second section. Here's a step-by-step approach to tackle this:

©2025 Codefinity. All Terms Reserved.

1. First, you need to **load the raw data** into the database. This involves **reading the CSV file** and populating the target table with all card data;
2. Once the data is in the database, ensure the **correct data types** are applied to each column (e.g., ensuring **numeric fields** like `credit_limit` are recognized as numeric, **date fields** like `acct_open_date` are formatted correctly, etc.);
3. After the data is loaded and formatted, you can perform a **Lookup** activity in **Azure Data Factory (ADF)** to identify all **distinct card types**. This will give you a list of **unique card types** present in the dataset;
4. Use a **ForEach** activity to process each **unique card type** separately. Inside the loop, you can **filter the data by card type**, ensuring that each card type has its own table;
5. For each card type, **create a separate table** in your database and **insert the relevant records** from the dataset.

By following these steps, you will be able to correctly **segregate** and **load the data into different tables** based on the **card type**.

©2025 Codefinity. All Terms Reserved.

codefinity

Solving Loading Data Challenge

I'll now take you through my solution to the previous challenge. Even if you have already solved the task, I strongly recommend watching the video. As a result, you will either learn an alternative approach to solving this task or identify and adjust any mistakes in your own approach.

Here are the scripts used in my solution:

Create Card Data Script

©2025 Codefinity. All Terms Reserved.

🕮 Challenge: Conducting Transaction Analysis

Financial institutions frequently assess the **creditworthiness** of their customers to enhance **risk management** and make better-informed decisions. By analyzing **debt levels** and **financial behavior** through user transactions, institutions can categorize customers into **risk groups**.

For example, individuals with a **high debt-to-income ratio**, indicating they owe significantly more than they earn, might be flagged as **Risk Users** due to their potential financial instability. In contrast, those with a healthier balance between their income and debt are classified as **Normal Users**, reflecting a lower likelihood of financial risk.

In the previous challenge, we loaded the **cards data**, and now the goal is to classify **credit card users** as either **Risk Users** or **Normal Users** based on their **custom debt-to-income ratio**.

To complete this task, you will need additional data: - **Transactions data** to calculate key metrics such as the **average transaction amount**, the **sum of all transactions**, and the **chip usage rate** (the ratio of transactions made using chips compared to the total number of transactions); - **Users data** to calculate the **custom debt-to-income ratio**, which is determined by dividing the total debt by the yearly income.

By using these datasets, you can evaluate users' financial behavior and classify them into the appropriate risk category.

Users Table

©2025 Codefinity. All Terms Reserved.

ID	Current Age	Retirement Age	Birth Year	Birth Month	Gender	Addres
825	53	66	1966	11	Female	462 Rose Lane
1746	53	68	1966	12	Female	3606 Federal Boulevar
1718	81	67	1938	11	Female	766 Third Drive
708	63	63	1957	1	Female	3 Madison Street
1164	43	70	1976	9	Male	9620 Valley Stream Drive

Transactions Table

©2025 Codefinity. All Terms Reserved.

codefinity

Transaction ID	Client ID	Card ID	Amount	Use Chip	Merchant ID
7475327	1556	2972	-$77.00	Swipe Transaction	59935
7475328	561	4575	$14.57	Swipe Transaction	67570
7475329	1129	102	$80.00	Swipe Transaction	27092
7475331	430	2860	$200.00	Swipe Transaction	27092
7475332	848	3915	$46.41	Swipe Transaction	13051
7475333	1807	165	$4.81	Swipe Transaction	20519
7475334	1556	2972	$77.00	Swipe Transaction	59935
7475335	1684	2140	$26.46	Online Transaction	39021
7475336	335	5131	$261.58	Online Transaction	50292

©2025 Codefinity. All Terms Reserved.

7475337	351	1112	$10.74	Swipe Transaction	3864
7475338	554	3912	$3.51	Swipe Transaction	67570
7475339	605	5061	$2.58	Swipe Transaction	75781

Transactions Dataset

Users Dataset

As the result of this task you will have two tables - one for risk users and one for normal users. They will look like this.

Risk Users

©2025 Codefinity. All Terms Reserved.

User ID	Sum of Transactions	Average Transaction	Chip Usage Rate	Debt-to-Income Ratio
201	$1,500.00	$50.00	60	55
310	$1,800.00	$45.00	80	65
487	$2,500.00	$62.50	75	60
910	$2,800.00	$70.00	85	70
1452	$3,200.00	$80.00	65	75

Normal Users

©2025 Codefinity. All Terms Reserved.

User ID	Sum of Transactions	Average Transaction	Chip Usage Rate	Debt-to-Income Ratio
2001	$1,200.00	$40.00	50	40
3504	$1,600.00	$50.00	60	45
2115	$2,000.00	$55.00	55	35
1120	$1,500.00	$45.00	65	30
1309	$2,100.00	$70.00	75	50

Algorithm Description

To solve this task, you can use the materials from the third section. Here's a step-by-step guide on how to do the task:

©2025 Codefinity. All Terms Reserved.

1. First, **load the raw data** into the database. This involves reading the CSV files for **users**, **transactions**, and **cards** and populating the respective tables. To do it you should use **Script** and **Copy** activities just like in the previous challenge;
2. After loading, ensure the **correct data types** are applied by using another **Script** activity;
3. Filter the **users table** to include only users with **credit cards**. This can be done by joining the **users** and **cards** tables on `client_id`. You should use that **cards** table that stores only credit cards data (you have created this table in the previous challenge);
4. Calculate the **Credit Score Ratio** for the filtered users by dividing `total_debt` by `yearly_income`. Based on this ratio:
5. Classify users with a ratio > 50% as **Risk Users**;
6. Classify users with a ratio ≤ 50% as **Normal Users**;
7. Aggregate the **transactions data** for each user group (**Risk Users** and **Normal Users**) to compute the following metrics:
8. **Total Transaction Amount**: Sum of all transactions;
9. **Average Transaction Amount**;
10. **Chip Usage Ratio**: Proportion of chip transactions to total transactions;
11. Create separate tables for **Risk Users** and **Normal Users** in the database and populate them with the aggregated metrics, ensuring all required fields (e.g., `client_id`, metrics) are included.

By following these steps, you can efficiently process the data and create the required outputs using only dataflow activities. Good luck!

©2025 Codefinity. All Terms Reserved.

codefinity

Note

There are many different ways to complete the task – you can try using other activities or different sequences of activities. Feel free to experiment!

©2025 Codefinity. All Terms Reserved.

Solving Conducting Analysis Challenge

Let's now look at one possible solution for the previous challenge:

Here are the scripts used to load raw data and transform it using **Script** activity:

Create User and Transaction Data Script

©2025 Codefinity. All Terms Reserved.